

Linearna Regresija

```
#Neophodni importi
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import
variance_inflation_factor as vif
import statsmodels.formula.api as smf
from sklearn.model_selection import train_test_split
from statsmodels.stats.outliers_influence import
variance_inflation_factor
from scipy.stats import shapiro

#Ucitavanje fajla
df = pd.read_csv('./podaci/fastfood.csv',
na_values=['NA'])

#Prikazivanje
display(df.head())
print(df.info())

#Broji koliko ima NA vrednosti u svakom atributu
df.isna().sum()

#Na osnovu datih informacija, vidimo da atributi
fiber,protein i calcium imaju NA vrednosti
# atributi restaurant, item, cal_fat i protein su tipa
string. u ovom obliku mozemo ostaviti item i restaurant
jer nam nisu bitni i po prirodi su nenumericke vrednosti

#Pravimo podskup na osnovu uslova (ne sadrzi proizvode
gde je total_fat>125)
df_subset = df.loc[df['total_fat']<=125].copy()
#Ispisujemo broj redova za skup i podskup
print(f"Broj redova u df_subset: {len(df_subset)}")
print(f"Broj redova u df: {len(df)}")

#Pravimo neki podskup sa na vrednostima
special_missing = ['- ', ' ', '']
```

```
#Proverimo koje kolone PODSKUPA imaju na vrednosti  
print(df_subset.isin(special_missing).sum())  
#To su restaurant, cal_fat, protein  
#Menjamo specijalne karaktere  
df_subset.replace(special_missing, np.nan, inplace=True)  
#Ponovna provera kojom vidimo da smo ih uspesno izbacili  
print(df_subset.isin(special_missing).sum())
```

```
#Pretvaramo cal_fat i protein u numericke  
df_subset['protein'] =  
pd.to_numeric(df_subset['protein'], errors='coerce')  
df_subset['cal_fat'] =  
pd.to_numeric(df_subset['cal_fat'], errors='coerce')
```

```
#Proveravamo jel su promenjeni  
print(df_subset.info())
```

```
#Pravimo subset gde su samo numericki podaci  
df_numeric = df_subset.select_dtypes(include =  
['number']).copy()  
print(df_numeric.info()) #Ovom proverom vidimo da nemamo  
vise attribute tipa str
```

```
#Sad proveravamo NA vrednosti sa procentima  
df_numeric.isna().sum()  
print((df_numeric.isna().mean() * 100).round(2))  
#Primecujemo da samo atributi cal_fat, fiber, protein i  
calcium nemaju vrednost 0.00  
#Ako je vrednost atributa veka od 20%, izbacujemo ga  
skroz, a ako je manja onda ga menjamo medijanom  
#Posto je calcium ima dosta NA, one nece biti zamenjene  
vec ce se on izbaciti  
df_numeric.drop(columns='calcium', inplace=True)  
df_numeric.info()  
# Deskriptivna statistika:  
display(df_numeric.describe())
```

```
#Ostale redove menjamo medijanom (Shapiro)
```

```
#1. pravimo podskup sa NA
cols_with_na = ['cal_fat', 'protein', 'fiber']
#2.provera normalne raspodele
for col in cols_with_na:
    series = df_numeric[col].dropna()
    stat,p = shapiro(series)
    print(f"{col} : p {p:.5f} - {'nije normalna' if p <
0.05 else 'normalna'} distribucija")
```

*#3. ako nije norm onda menjamo medijanom a ako jeste
onda srednjom vrednoscu*

#4. posto jeste onda menjamo medijanom

```
for col in cols_with_na:
    median = df_numeric[col].median()
    df_numeric[col] = df_numeric[col].fillna(median)
```

#5. provera jel smo uspesno uklonili NA

```
df_numeric.isna().sum()
```

Korelaciona matrica

```
corr_matrix = df_numeric.corr(numeric_only=True).round(2)
plt.figure(figsize=(10, 8))
sb.heatmap(corr_matrix, annot=True, cmap='coolwarm',
vmin=-1, vmax=1)
plt.title("Korelaciona matrica")
plt.tight_layout()
plt.show()
print(df_numeric[['total_fat', 'protein']].head(20))
#Na osnovu kor matrice i definisanog targeta protein
gledamo koje su vece od 0.5 bilo da su + ili -
```

#Izbor prediktora

```
target = 'protein'
```

#Feature su prve tri najvece a da su vece od 0.5

```
features = ['cholesterol', 'calories', 'sodium']
```

*#Na korelacionoj matrici se moze primetiti da su za
predvidjanje atributa protein najznacajniiji atributi:*

*#a) cholesterol 0.87 – jedinичno povećanje atributa
cholesterol dovodi do povećanja atributa protein za 0,87*

*#b) calories 0.8 – jedinичno povećanje atributa calories
dovesce do povećanja atributa protein za 0.8*

#c) sodium 0.76 – jedinичno povećanje atributa sodium dovede do povećanja atributa protein za 0.76

```
#Podela na train i test
train,test = train_test_split(df_subset, test_size=0.2,
random_state=123) #obicno je 42
#Provera train i test
print(f"Broj redova u train: {len(train)}")
print(f"Broj redova u test: {len(test)}")
```

```
#Pravljenje modela LR
#Prvo navodio target pa onda feature
lm1 = smf.ols('protein ~ cholesterol + calories +
sodium', data=train).fit()
print(lm1.summary())
```

#Reziduali predstavljaju razliku između stvarnih i predviđenih vrednosti ciljne promenljive protein i idealno je da su oni raspoređeni oko nule

#Medijanu računamo tako što od stvarne vrednosti oduzmemo predviđenu

#R-squared je koef determinacije i on iznosi 0.852 tj model objašnjava oko 85.2% varijabilnosti ciljne promenljive

#F-statistika je visoka(778)

#Statistika značajnosti ($P > |t|$):
#cholesterol i sodium imaju p-vrednosti < 0.001 što znači da su vrlo značajni prediktori, dok calories ima 0.015 i on je statistički značajan

#Intercept(presretanje) iznosi 2.5108 i ono predstavlja vrednost kada je lstat=0

#Koeficijenti:
#cholesterol: 0.1970 – povećanje za 1 jedinicu dovodi do rasta proteina za -0.197

```
#calories: 0.0057 – povecanje za 1 jedinicu dovodi do  
rasta proteina za -0.0057  
#sodium: 0.0066 – povecanje za 1 jedinicu dovodi do rasta  
proteina za -0.0066
```

```
#Predikcija na test skupu  
y_true = test['protein']  
X_test = test[['cholesterol', 'calories', 'sodium']]  
y_pred = lm1.predict(X_test)
```

```
#Pravimo DataFrame sa poredjenjem  
results = pd.DataFrame({  
    'Stvarno': y_true.values,  
    'Predvidjeno': y_pred.round(2),  
})
```

```
#Prikaz prvih 10 predikcija  
print(results.head(10))  
#Na osnovu prikazanih informacija vidimo koliko se koji  
razlikuje u odnosu na stvarnu vrednost
```

```
#Dijagnosticki grafikoni, sve iz helpera  
# Residuals vs Fitted  
#Grafikon prikazuje raspodelu reziduala (razlike izmedju  
stvarnih i predvidjenih vrednosti) u odnosu na  
predvidjenje vrednosti  
#Koristi se da proveriti da li je zadovoljena pretpostavka  
linearne veze  
#Reziduali bi trebalo da budu rasporedjeni oko  
horizontalne linije(vrednost bliska nuli)  
#Sa grafikona se moze zakljuciti da postoje manja  
odstupanja sto moze ukazivati na neadekvatnost modela
```

```
# Normal Q-Q (širi graf da linija deluje blaže)  
#Prikazuje da li reziduali imaju norm raspodelu  
#Tacke bi trebalo da leze duz dijagonale  
#Posto tacke odstupaju od dijagonale na krajevima,  
pretpostavka normalnosti nije u potpunosti ispunjena
```

```
# Scale-Location  
#Prikazuje da li je varijabilnost reziduala konstantna  
duz predikcija
```

```
#Idealno bi crvena linija trebala da bude horizontalna i  
da tacke budu ravnomerno rasporedjene  
# U ovom slucaju varijansa reziduala nije potpuno  
konstantna sto ukazuje na mogucu HETEROSKEDASTICNOST
```



```
# Residuals vs Leverage + Cook's D threshold  
#Identifikuje ekstremno visoke/niske vrednosti (outliere)  
i njihove uticaje na model  
#Vecina tacaka bi trebalo da bude unutar granica, a  
ekstremne vrednosti retke  
#Na osnovu Cookove distance vidi se da postoji nekoliko  
potencijalnih outliera, sto moze uticati na stabilnost  
modela
```

#Provera multikolinearnosti

```
X = sm.add_constant(train_df[features])  
vif = pd.Series([variance_inflation_factor(X.values, i) for i  
in range(X.shape[1])], index=X.columns)  
print('√VIF prve verzije:\n', np.sqrt(vif))
```

*#S obzirom da je za atribut calories sqrt(vif) vrednost >2,
može se zaključiti da postoji multikorelisanost, koja može
dovesti do nepouzdanih procena.*

*#Novi model za predikciju proteina treba kreirati sa
atributima cholesterol i sodium.*

```
lm2 = smf.ols(f'{target} ~ cholesterol + sodium',  
data=train_df).fit()
```

Provera VIF za novi model

```
X = sm.add_constant(train_df[['cholesterol', 'sodium']])  
vif = pd.Series([variance_inflation_factor(X.values, i) for i  
in range(X.shape[1])], index=X.columns)  
print('√VIF druge verzije:\n', np.sqrt(vif))  
#Multikolienearnost ne postoji više.
```

Stablo odlucivanja

```
# Uključivanje potrebnih biblioteka
import pandas as pd
import numpy as np
import seaborn as sb
import matplotlib.pyplot as plt
from matplotlib.pyplot import xlabel
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,
plot_tree
from sklearn.metrics import confusion_matrix

#ucitavanje pod
df = pd.read_csv('./podaci/bank.csv', na_values=['-', ' ', ''])
display(df.head())
df.info()
df.isna().sum()

#Provera da li ima na u exited
print(df['Exited'].isna().sum())
#pravimo novi atribut koji je yes kad je exited 0 a no kad
je nes drugo
df['Stayed'] = np.where(df['Exited'] == 0, 'Yes', 'No')
#izlazna varijabla se prebacuje u 0 i 1
df['Stayed'] = df['Stayed'].map({'Yes':1, 'No':0})
#uklanjamo exited
df.drop(columns=['Exited'], inplace=True)

#Kreiranje podskupa
print(df['Age'].isna().sum())
#Zamena medijanom
df['Age'].fillna(df['Age'].median())
#postavljanje uslova
df = df[df['Age'] <= 87]
print(df['Age'].describe())
#provera na vr
print(df.isna().sum())
```

```
#vidimo da na imaju surname i geography i surname cemo  
izbaciri jer nam nije bitna info  
df.drop(columns=['Surname'], inplace=True)
```

```
#Sad atribut geography menjamo najcescom vr  
df['Geography'] =  
df['Geography'].fillna(df['Geography'].mode()[0])  
##popunjavanje sa onom koja je najcesca  
df['Geography'].isna().sum()
```

```
#Sad crtamo grafike za svaki atribut i ako je num bez  
konkretnih definisanih vr onda radimo sa sb.kdeplot pa  
ako je raspodele gotovo potpuno preklapaju dropujemo ga  
CreditScore
```

```
plt.figure()  
sb.kdeplot(data=df, x='CreditScore', hue='Stayed',  
fill=True)  
plt.show()
```

```
#Ne treba ga zadržati jer se raspodele za varijablu  
Stayed gotovo potpuno preklapaju
```

```
df.drop(columns=['CreditScore'], inplace=True)
```

```
#Ako je u pitanju num vr ali sa nekom konkretnom npr 0 i  
1 onda radimo sa sb.countplot  
NumOfProducts
```

```
plt.figure()  
sb.countplot(data=df, x='NumOfProducts', hue='Stayed')  
plt.xlabel('Number of Products')  
plt.ylabel('Broj klijenata')  
plt.show()
```

```
#Bar plotovi za Geography, Gender i Card.Type grupisani  
po Stayed (tipa str)
```

```
for col in ['Geography', 'Gender', 'Card.Type']:  
    plt.figure()  
    sb.countplot(data=df, x=col, hue='Stayed')  
    plt.xlabel(col)  
    plt.ylabel('Broj klijenata')  
    plt.legend(title='Stayed')  
    plt.tight_layout()  
    plt.show()
```



```

#izbacujemo Card.Type
df.drop(columns=['Card.Type'], inplace=True)

#menjamo geography i gender u num
#selektujemo koje su str
df_vars =
df.select_dtypes(include='str').columns.tolist()
print(df_vars)

for col in df_vars:
    print(f"{col} : {df[col].nunique()}")

df.Geography.value_counts()

df.Gender.value_counts()

#Sad je potrebno da importujemo OHE da bismo prenacili u
num
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False, drop='first')
#uklanja prvu kategoriju
cats = ['Geography', 'Gender']
encoded_array = ohe.fit_transform(df[cats])
encoded_cols = ohe.get_feature_names_out(cats)

df_ohe = pd.DataFrame(encoded_array,
columns=encoded_cols, index = df.index)
df = df.drop(columns=cats).join(df_ohe)

df.head()

#Podela na train i test
x = df.drop(columns='Stayed')
y = df['Stayed']
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.2, random_state=42, stratify=y)
#Raspodela klasa u trening i test skupu
print(y_train.value_counts(normalize=True))
print(y_test.value_counts(normalize=True))
#Upoređujemo proporciju prvog i drugog tj traina i testa
i pos je isto znaci da smo sve dobro uradili

```

```
#Izracunavanje optimalne vrednosti paramtra ccp_alpha
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV,
StratifiedKFold
```

```
cv = StratifiedKFold(n_splits=10, shuffle=True,
random_state=42)
param_grid = {'ccp_alpha':np.arange(0.0, 0.05, 0.0025)}
grid =
GridSearchCV(DecisionTreeClassifier(random_state=42),
param_grid = param_grid, cv=cv, scoring='accuracy')
grid.fit(x_train, y_train)
```

```
best_ccp = grid.best_params_['ccp_alpha']
print(best_ccp)
```

```
#Kreiranje modela stabla odlucivanja
tree =
DecisionTreeClassifier(ccp_alpha=best_ccp,random_state=42
)
tree.fit(x_train, y_train)
```

```
#Prikaz
plt.figure(figsize=[30,10])
plot_tree(tree, feature_names=x_train.columns,
class_names=['No', 'Yes'], filled=True, rounded=True,
fontsize=10)
plt.show()
```

```
#Kreiranje predikcije za test skup
y_pred = tree.predict(x_test)
```

```
#Kreiranje matrice konfuzije
```

	<i>#Predicted Positive</i>	<i>#Predicted</i>
<i>Negative</i>		
<i>#Actual Positive</i>	<i>TP</i>	<i>FN</i>
<i>#Actual Negative</i>	<i>FP</i>	<i>TN</i>

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
#Kreiranje DataFrame sa oznakama redova i kolona
cm_df = pd.DataFrame(cm, index= ['Stvarno:No',
                                'Stvarno:Yes'],
                    columns=['Predvidjeno:No',
                            'Predvidjeno:Yes'])
print(cm_df)
```

#4 vrednosti postoje u matrici konfuzije TP, TN, FP i FN:
True-Positive(TP) – koliko klijenata smo predvideli da će ostati u banci i oni su zaista ostali – 745
True-Negative(TN) – koliko klijenata smo predvideli da će napustiti banku i oni su je zaista napustili – 92
False-Positive(FP) – koliko klijenata smo predvideli da će ostati u banci, a oni su je ipak napustili – 112
False-Negative(FN) – koliko klijenata smo predvideli da će napustiti banku, a oni su ipak ostali – 30

#4 metrike se koriste za evaluaciju performansi klasifikacionog modela:

Accuracy (tačnost) – procenat tačnih predikcija (i pozitivnih i negativnih);
Precision (preciznost) – procenat tačno predviđenih pozitivnih predikcija u odnosu na sve pozitivne predikcije koje je model napravio;
Recall (odziv) – procenat pozitivnih predikcija koje je model tačno identifikovao kao pozitivne u odnosu na ukupan broj stvarno pozitivnih;
F1 – mera za balansiranje vrednosti precision-a i recall-a.

#Pravljenje funkcija za izr eval metrika

```
def eval_metrics(cm, model_name=""):
    from pandas import Series
    TP = cm[1,1]
    TN = cm[0,0]
    FP = cm[0,1]
    FN = cm[1,0]

    accuracy = (TP+TN)/(TP+TN+FP+FN)
    precision = TP/(TP+FP)
    recall = TP/(TP+FN)
    f1 = 2*precision*recall/(precision+recall)
```

```
    return Series(data=[accuracy, precision, recall, f1],  
index=["Accuracy", "Precision", "Recall", "F1"],  
name=model_name)
```

```
eval1 = eval_metrics(cm, 'Tree')  
eval1
```

#accuracy – koliko tačno predviđamo izlaznu varijablu. Od 979 clijenata tačno je predviđeno 837, odnosno, za 85.5% clijenata je tačno predviđeno da li će ostati u banci ili ne.

#precision – od svih clijenata predviđenih da će ostati u banci, koji procenat clijenata je zaista ostao. Od svih clijenata za predviđenih da će ostati u banci, 87% njih je stvarno ostalo.

#recall – od svih clijenata koji su zaista ostali u banci, koji procenat clijenata je predviđeno da će ostati u banci. Među svim klijentima koji su zaista ostali u banci, tačno je predviđeno 96%.

#F1 – predstavlja meru performansi modela za balansirane vrednosti precision i recall. S obzirom da su precision i recall u antagonističkom odnosu, povećanje jedne dovodi do smanjenja druge i obrnuto. Kako su i precision i recall visoki, vrednost ove metrike je visoka 0.91. U kontekstu zadatka, ova metrika pokazuje da nismo pravili mnogo grešaka predviđajući ostanak clijenata u banci

kNN

```
import pandas as pd
import numpy as np
from scipy.stats import shapiro
from sklearn.neighbors import KNeighborsClassifier
import seaborn as sb
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay, accuracy_score,
classification_report, precision_score, recall_score,
f1_score
from sklearn.preprocessing import RobustScaler

# Učitavamo podatke (navodimo koje vrednosti ćemo
# smatrati nedostajućim: '-', ' ', '')
df = pd.read_csv("./podaci/spotify.csv", na_values=['-',
' ', ''], encoding='latin1')

display(df.head())
df.info()

#Da bismo videli info za Q3
display(df.describe())
# Proveravamo da li varijabla streams ima nedostajuće
vrednosti jer na osnovu nje kreiramo izlaznu varijablu
print(df['streams'].isna().sum())
#Nema
#Racunamo treci kvartil Q3
streams_q3 = df['streams'].quantile(0.75)
print(streams_q3)

#Kreiramo izlaznu varijablu HighStreams na osnovu
streams
df['HighStreams'] = np.where(df['streams'] > streams_q3,
'Yes', 'No')
#Mapiranje
df['HighStreams'] = df['HighStreams'].map({'Yes': 1,
'No': 0})
#Izbacujemo varijablu streams jer smo je koristili za
kreiranje izlazne varijable !!!
```

```

df.drop(columns=['streams'], inplace=True)

#Nedostajuće vr
print(df.isna().sum())
#in_spotify_charts i in_apple_charts po 2

#Varijable in_spotify_charts i in_apple_charts su
numeričkog tipa i imaju mali broj nedostajućih vrednosti.
Zbog toga ćemo koristiti metod zamene prosečnom vrednošću
(mean ili median). Da bismo odlučili da li da koristimo
mean ili median, prvo ćemo proveriti raspodelu ovih
varijabli pomoću Shapiro-Wilk testa. Ako varijable imaju
normalnu raspodelu, nedostajuće vrednosti ćemo zameniti
mean-om, a ako nemaju normalnu raspodelu, koristićemo
median.

cols_with_na = ['in_spotify_charts', 'in_apple_charts']
#Shapiro test
for col in cols_with_na:
    series = df[col].dropna()
    stat, p = shapiro(series)
    print(f"{col} : p {p:.5f} - {"nije normalna" if p <
0.05 else " normalna"}")
#Obe varijable nemaju normalnu raspodelu pa se nedostajuće vr
menjaju medijanom
for col in cols_with_na:
    median = df[col].median()
    df[col] = df[col].fillna(median)

#Provera Na vr vidimo da su sve uspesno zamenjene
print(df.isna().sum())

#Procena znacajnosti atributa za ukljucivanje u model
df.info()

#Opet crtamo grafike preko sb.kdeplot i sb.countplot i
izbacujemo
plt.figure()
sb.kdeplot(data=df, x='released_year', hue='HighStreams',
palette='Blues_d')
plt.show()

#Skaliranje numerickih varijabli

df.drop(columns=['track_name'], inplace=True)

```

```
print(df.describe())
print(df.info())
```

```
numeric_vars = ['released_year', 'in_spotify_playlists',
                'in_spotify_charts', 'in_apple_playlists',
                'in_apple_charts']
```

#Proveravamo da li numeričke varijable imaju autlajere kako bismo odlučili da li ćemo koristiti normalizaciju ili standardizaciju za skaliranje podataka. Ako varijable nemaju autlajere, koristićemo normalizaciju, a ako imaju, koristićemo standardizaciju.

#Funkcija za izracunavanje broja autjalera

```
def count_outliers(variable):
    q1 = variable.quantile(0.25)
    q3 = variable.quantile(0.75)
    iqr = q3 - q1    #interkvartilna razlika
    lower = q1 - 1.5 * iqr #donja granica
    upper = q3 + 1.5 * iqr #gornja granica
    return ((variable<lower) | (variable>upper)).sum()
```

```
outliers_per_column =
df[numeric_vars].apply(count_outliers)
print(outliers_per_column)
#Sve varijable imaju autlajere tkd cemo koristiti
STANDARDIZACIJU
```

#Proveravamo da li varijable imaju nromalnu raspodelu kako bismo odlucili na koji nacin da izvorsimo standardizaciju

```
shapiro_results = df[numeric_vars].apply(lambda x:
shapiro(x)[1])
print(shapiro_results)
#Sve vrednosti su manje od 0.05 tkd nemaju norm raspodelu
pa cemo ih standardizovati pomocu RobustScalera koji
koristi median i iqr
```

#Standardizujemo num var pomocu RobustScalera

```
robust_scaler = RobustScaler()
df_st = pd.DataFrame() #Kreiramo praznan df u koji cemo
ubaciti sve vresnoti pripremljene za knn
```

```

df_st[numeric_vars] =
robust_scaler.fit_transform(df[numeric_vars])

display(df_st.head())

#Transformacija binarnih i kategorijskih u numericke
print(df.info())
#samo je mode str
print(df['mode'].unique())
# Varijabla mode ima samo dve vrednosti ('Major' i
'Minor'), binarna je i možemo je mapirati u brojeve (npr.
0 i 1)
df_st['mode'] = df['mode'].map({'Major': 0, 'Minor': 1})

#dodajemo izlaznu var u standardizovani df
df_st['HighStreams'] = df['HighStreams']

#Test i trening
x = df_st.drop(columns=['HighStreams'])
y = df_st['HighStreams']
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.2, random_state=42, stratify=y)

#izracunavanje optimalne vrednosti za parametar k

knn = KNeighborsClassifier()
# Koristimo neparne vrednosti za k (broj suseda) u opsegu
od 3 do 25 kako bi uvek postojala dominantna klasa
param_grid = {'n_neighbors': list(range(3, 26, 2))}
#kreiranje kros validacije za 10 iter
cv = GridSearchCV(estimator=knn, param_grid=param_grid,
scoring='accuracy', cv=10, verbose = 1, n_jobs=-1)
cv.fit(x_train, y_train)

best_k = cv.best_params_['n_neighbors']
print(best_k) #Optimalna vrednost za param k

#Kreiranje knn klasifikacionog modela
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(x_train, y_train)

```



```
#Predikcije  
y_pred = knn.predict(x_test)
```

```
#Kreiranje matrice konfuzije  
cm_knn = confusion_matrix(y_test, y_pred)  
disp = ConfusionMatrixDisplay(confusion_matrix=cm_knn,  
display_labels=['No', 'Yes'])  
disp.plot()  
plt.show()
```

#True positive (TP) – za 34 pesama smo predvideli da će imati veliki broj puštanja i one zaista imaju veliki broj puštanja

#True negative (TN) – za 137 pesama smo predvideli da neće imati veliki broj puštanja i one zaista nemaju veliki broj puštanja

#False positive (FP) – za 6 pesama smo predvideli da će imati veliki broj puštanja, a one zapravo nemaju veliki broj puštanja

#False negative (FN) – za 14 pesama smo predvideli da neće imati veliki broj puštanja, a one zapravo imaju veliki broj puštanja

#Na glavnoj dijagonali matrice (34 i 137) nalaze se tačne predikcije, dok se na sporednoj dijagonali (6 i 14) nalaze pogrešne predikcije.

#Evaluacione metrike

#4 metrike koje se najčešće koriste za procenu uspešnosti klasifikatora su:

#Accuracy (tačnost) – procenat tačnih predikcija (i pozitivnih i negativnih) u odnosu na sve observacije

#Precision (preciznost) – procenat observacija za koje smo predvideli da su pozitivne i koje zaista jesu pozitivne

#Recall (odziv) – procenat observacija koje su stvarno pozitivne i koje smo mi tačno predvideli kao pozitivne

#F1 score – mera za balansiranje vrednosti precision-a i recall-a

#Pravljenje evalucionih metrika

```
def compute_eval_metrics(y_true, y_pred):  
    accuracy = accuracy_score(y_true, y_pred)
```

```
precision = precision_score(y_true, y_pred,  
pos_label=1)  
recall = recall_score(y_true, y_pred, pos_label=1)  
f1 = f1_score(y_true, y_pred, pos_label=1)  
return accuracy, precision, recall, f1
```

```
knn_eval1 = compute_eval_metrics(y_test, y_pred)  
print(knn_eval1)
```

#Tumačenje metrika u kontekstu problema koji se rešava u zadatku:

#Accuracy – za 89.53% pesama smo tačno predvideli da li će imati veliki broj puštanja ili ne
#Precision – od svih pesama za koje smo predvideli da će imati veliki broj puštanja, 85% zaista ima veliki broj puštanja
#Recall – od svih pesama koje zaista imaju veliki broj puštanja, mi smo tačno predvideli 70.83% njih
#F1 score – mera koja balansira preciznost i odziv, i u našem slučaju iznosi 0.77. Može se reći da naš model uspešno balansira između sposobnosti da prepozna većinu pesama koje zaista imaju veliki broj puštanja i toga da ne označava previše pesama kao pesme sa velikim brojem puštanja kada one to zapravo nisu.

Neuronske mreze

```
#Neophodni importi
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from scipy.stats import shapiro
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay, accuracy_score, precision_score,
recall_score, f1_score, classification_report

# Učitavamo podatke (navodimo koje vrednosti ćemo smatrati
nedostajućim: '-', ' ', '')
df = pd.read_csv('./podaci/breast_cancer.csv',
na_values=['-', ' ', ''])
display(df.head())
df.info()

# Proveravamo da li varijable imaju nedostajuće vrednosti
print(df.isna().sum())
# Varijable mean radius i mean texture imaju po 2
nedostajuće vrednosti

#provera dal imaju norm rasp
cols_with_na = ['mean radius', 'mean texture']

for col in cols_with_na:
    series = df[col].dropna()
    stats, p = shapiro(series)
    print(f"{col} : p{p:.5f} - {'nije norm' if p < 0.05
else 'normalna'}")

# Obe varijable imaju vrednost p < 0.05 što znači da nemaju
normalnu raspodelu i nedostajuće vrednosti zamenićemo
medijanom
for col in cols_with_na:
    median = df[col].median()
    df[col] = df[col].fillna(median)

print(df.isna().sum()) # Nema više nedostajućih vrednosti
```

```
#Delimo podatke na trening i test  
X = df.drop(columns='isBenign')  
y = df['isBenign']  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2, random_state=42, stratify=y)
```

```
#standardizujemo varijable  
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

```
#Kreiranje neuronske mreže  
from IPython.display import display  
mlp1=MLPClassifier(hidden_layer_sizes=(64,32),  
max_iter=1000, random_state=42, solver='sgd')  
#Arhitektura kreirane neuronske mreže je 30-64-32-1.
```

```
#Ulazni sloj sadrži 30 neurona. koliko kolona ima  
#Mreža ima dva skrivena sloja:  
#prvi skriveni sloj sadrži 64 neurona,  
#drugi skriveni sloj sadrži 32 neurona.  
#Izlazni sloj sadrži 1 neuron. taj izlazni
```

```
#Trening neuronske mreže  
mlp1.fit(X_train, y_train)  
print(mlp1)  
display(mlp1)  
#narandzasti su ovi koje smo menjali
```

```
#Tokom treninga neuronske mreže izvršavaju se sledeći  
koraci:
```

```
#Forward propagation – ulazni podaci prolaze kroz mrežu i  
izračunava se predikcija modela.
```

```
#Računanje greške (loss) – predikcije modela porede se sa  
stvarnim vrednostima i izračunava se greška modela.
```

```
#Backward propagation – na osnovu izračunate greške  
određuje se kako svaki parametar mreže utiče na grešku.
```

#Ažuriranje parametara – težine i bias vrednosti mreže ažuriraju se pomoću optimizacionog algoritma sa ciljem smanjenja greške modela.

#Ovaj proces se ponavlja kroz više epoha, pri čemu jedna epoha predstavlja jedan prolazak kroz ceo trening skup podataka.

```
y_pred = mlp1.predict(X_test)
class_report = classification_report(y_test, y_pred)
print(class_report)
```

#Prikaz promene vrednosti fje greske tokom tr

```
plt.figure()
plt.plot(mlp1.loss_curve_)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid()
plt.show()
```

#kad nas model opada i stabilizuje se na niskoj vr onda on dobro uci

#kreiranje matrice konfuzije

```
y_pred = mlp1.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=['malignant', 'benign']) # labels su od ove
glavne
disp.plot()
plt.show()
```

#True positive (TP) – za 41 tumor smo predvideli da će biti maligni i one zaista jeste bio maligni

True negative (TN) – za 70 tumora smo predvideli da će biti benigni i oni zaista jesu bili benigni

False positive (FP) – za 2 tumora smo predvideli da će biti maligni, a oni su zapravo bili benigni

False negative (FN) – za 1 tumor smo predvideli da će biti benigni, a on je zapravo bio maligni

```
# Na glavnoj dijagonali matrice (41 i 70) nalaze se tačne  
predikcije, dok se na sporednoj dijagonali (2 i 1) nalaze  
pogrešne predikcije.  
# Evaluacione metrike  
# 4 metrike koje se najčešće koriste za procenu uspešnosti  
klasifikatora su:  
# Accuracy (tačnost) – procenat tačnih predikcija (i  
pozitivnih i negativnih) u odnosu na sve observacije  
# Precision (preciznost) – procenat observacija za koje smo  
predvideli da su pozitivne i koje zaista jesu pozitivne  
# Recall (odziv) – procenat observacija koje su stvarno  
pozitivne i koje smo mi tačno predvideli kao pozitivne  
# F1 score – mera za balansiranje vrednosti precision-a i  
recall-a
```

```
def eval_metrics(y_true, y_pred):  
    accuracy = accuracy_score(y_true, y_pred)  
    precision = precision_score(y_true, y_pred,  
pos_label=0)  
    recall = recall_score(y_true, y_pred, pos_label=0)  
    f1 = f1_score(y_true, y_pred, pos_label=0)  
    return accuracy, precision, recall, f1
```

```
# Izračunavanje vrednosti evaluacionih metrika  
eval1 = eval_metrics(y_test, y_pred)  
print(eval1)
```

kMeans klasterizacija

```
#Neophodni importi
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
import sklearn
from sklearn import cluster
from sklearn.cluster import KMeans

#Ucitavanje podataka
df = pd.read_csv('./podaci/spotify2.csv', na_values=['-',
', ', ''])
df.info()
display(df.head())

#Pravimo novi dataset koji sadrži samo pesme čiji
speechiness je manji od ili jednak 60
df_sub = df[df['speechiness'] <= 60]
# Bacimo pogleda na podatke koje imamo
df_sub
df_sub.describe()

## Da vidimo kog tipa je svaka promenljiva
df_sub.info()

# Promenljiva in_spotify_charts je tipa object (zapravo se
tretira kao String), a trebalo
# bi da bude broj ali to ćemo naknadno videti zašto

# Gledamo deskriptivne statistike za numeričke promenljive
df_sub.describe()

# Izdvaja se to da instrumentalness uglavnom sadrži nule
(sve do 75 kvartila), pa je pitanje
# da li zapravo ima smisla koristiti tu promenljivu
#0sim toga, released_year uglavnom čine pesme iz raspona
2020–2023 (25 kvartil do max) i pitanje
# je da li to kao informacija doprinosi boljoj
klasterizaciji

# Razmatramo sledeće promenljive za izbacivanje:
```

```
# track_name – nije nam bitan naziv pesme radi
klasterizacije, a i nije numerička promenljiva

# mode – nije numerička, već kategorijska promenljiva.
Pošto ima samo dve vrednosti (Major i Minor)
# transformisaćemo je u binarnu (numeričku) varijablu
mapiranjem u sledećem koraku (npr. 0 – Major,
# 1 – Minor) i upotrebićemo je u klasterizaciji – nećemo je
izbaciti.

# Za promenljivu instrumentalness, uglavnom važi da je čine
nule (o do 75 kvartila), pa nju izbacujemo

# Ostale promenljive su numeričke (osim in_spotify_charts
koja je tipa object, a trebalo
# bi da bude broj ali to ćemo naknadno videti zašto u
sledećem koraku) tako da one ostaju – za sada.
df_sub.drop(columns=['track_name', 'instrumentalness'],
inplace=True)

# Pretvaramo kolonu in_spotify_charts u numeričku i menjamo
pogrešno unete vrednosti ('-', "" ili '_' ili tj. sve što
nije broj) sa NA vrednostima
df_sub['in_spotify_charts'] =
pd.to_numeric(df_sub['in_spotify_charts'], errors='coerce')

# Kolona mode jedina nije numerička, ali nju ćemo sada
pretvoriti u
# binarnu (numeričku) varijablu mapiranjem u sledećem
koraku (npr. 0 – Major,
# 1 – Minor)
df_sub['mode'] = df_sub['mode'].map({'Major': 0, 'Minor':
1})
df_sub.info()
df_sub['mode'].describe()
df_sub.isna().sum()
# Sa obzirom na to da ukupno imamo 4 nedostajuće vrednosti
na ceo dataset od 952 instance
# ok je da izbacimo te četiri instance (četiri cela reda)
jer ne to bi trebalo da utiče mnogo na
# rezultate klasterizacije

# mozemp da brisemo sve redove sa na
#df_sub.dropna(inplace=True)
```



```
#df_sub.isna().sum()
```

```
#isto tako mozemo da zamenimo ove 4 nedostajuce vr sa medijanom
```

```
df_sub.loc[df_sub['in_spotify_charts'].isna(),  
            'in_spotify_charts'] = df_sub['in_spotify_charts'].median()  
df_sub.loc[df_sub['in_apple_charts'].isna(),  
            'in_apple_charts'] = df_sub['in_apple_charts'].median()  
print(df_sub.isna().sum())  
df_sub.describe()
```

```
# Ovde sada postoji jedan veliki problem sa kodiranjem podataka koji će da utiče na klasterizaciju  
# a odnosi se na ove dve kolone: in_spotify_charts i in_apple_charts. Ako se pesma ne pojavljuje  
# na top listi, stoji vrednost 0. Najbolje pesme su na 1, 2 i 3 mestu a najgore na 147 odnosno 275  
# mestu (max) tj. što je niži broj, pesma je bolja.  
# – Ako se ostave ove nule, dobija se iskrivljena slika da su to NAJBOLJE pesme (0 je manje od 1, 2, 3...)  
# – Ako se zamene medijanom za tu kolonu, dobija se iskrivljena slika da su to osrednje pesme.
```

```
#  
# Možda je najbolje da se ove vrednosti  
# zamene sa max vrednošću za tu kolonu da bi se dobila realna slika o poziciji tih pesama na top listama.  
df_sub.loc[df_sub["in_apple_charts"]==0, "in_apple_charts"]  
= df_sub["in_apple_charts"].max()  
df_sub.loc[df_sub["in_spotify_charts"]==0,  
            "in_spotify_charts"] = df_sub["in_spotify_charts"].max()  
df_sub.describe()
```

```
# Sve nule iz ove dve kolone su zamenjene max vrednostima
```

```
#Korelaciona matrica
```

```
corr_matrix = df.corr(numeric_only=True).round(2)  
plt.figure(figsize=(10, 8))  
sb.heatmap(df_sub.corr(numeric_only=True), annot=True,  
            cmap='coolwarm', vmin=-1, vmax=1)  
plt.title("Korelaciona matrica")  
plt.tight_layout()  
plt.show()
```

```
#U korelaciji su in_apple_streams, in_spotify_playlists i streams i to puno utice na rez klasterizacije pa izbacujemo bilo koje dve mi cemo izbaciti in_apple_playlists i streams
df_sub.drop(columns=['in_apple_playlists', 'streams'], inplace=True)
```

```
#Proveravamo outliere po kolonama jer i oni uticu na klasterizaciju
# ako outliera ima preko 10% izbacujemo celu kolonu, a ako nema puno onda radimo winsorize
```

```
#uvodimo pomocnu fju draw_boxplot koja crta i prikazuje boxplot
```

```
def draw_boxplot(podaci, naslov):
    plt.boxplot(podaci)
    plt.xlabel(naslov)
    plt.show()
```

```
from scipy.stats.mstats import winsorize
#trazimo outliere, kolona in_spotify_playlists
draw_boxplot(df_sub['in_spotify_playlists'], 'in_spotify_playlists')
```

```
#ima outliere samo na gornjoj granici, winsorize u rasponu 0-90 ne otklanja outliere
winsorized_data = winsorize(df_sub['in_spotify_playlists'], limits=[0.00, 0.10])
```

```
#ako su samo gore outlieri, da su i gore i dole bilo bi 0.10,0.10 a da su samo dole onda 0.10,0.00
```

```
draw_boxplot(winsorized_data, 'Winsorized in_spotify_playlists')
plt.show()
```

```
#Zakljucak je da vise od 10% vrednosti ove prom cine outlieri i treba je izbaciti iz subeta
df_sub.drop(columns=['in_spotify_playlists'], inplace=True)
```

```
#Tako radimo za svaki i ako se desi da ima I dalje outliera ali ih je malo tj manje od 10% vr onda treba te vr zameniti sa winsorizovanim
```

```
df_sub['bpm'] = winsorized_data
```

```
#posle izbacivanja na vrednosti, sredjivanja i izbacivanja
promenljivih sa korelacijama i outlierima
df_sub.describe()
# vidimo 5 prom koje dolaze u obzir za klasterizaciju
```

```
#skaliranje (normalizacija) jer nasi podaci vise nemaju
outliere
```

```
from sklearn.preprocessing import MinMaxScaler
#promenljive su u veoma razl rasponima, potrebna je
normalizacija na skalu 0-1
#koristimo minmaxscaler koji vraca numpy array koji
pretvraamo nazad u dataframe
temp_data = MinMaxScaler().fit_transform(df_sub)
df_sub_scaled = pd.DataFrame(temp_data,
columns=df_sub.columns)
#proveravamo da li je sve skalirano kako treba
df_sub_scaled.describe()
```

```
#Primenom Elbow metode utvrditi najbolju vrednostr za broj
klastera
```

```
wcss = [] #within cluster sum of squares
```

```
#probamo od 1 do 10 klastera
```

```
for i in range(1, 11):
    kmeans = cluster.KMeans(n_clusters=i, init= 'k-means++',
max_iter=100, n_init=1000,
random_state=0).fit(df_sub_scaled)
    wcss.append(kmeans.inertia_)
```

```
plt.plot(range(1, 11), wcss)
plt.xlabel("Number of clusters")
plt.ylabel("WCSS")
plt.show()
wcss
```

```
#"Lakat" se pojavljuje kod 2 klastera i kod 4 klastera
drugi lakat(vidi se razlika \_)
```

```
#Izvršiti klasterizaciju za izabranu vrednost za k
```

```
#Radimo Kmeans klasterizaciju sa dva klastera i kmeans++
metodom za inicijalizaciju tezista
```

```
kmeans = cluster.KMeans(n_clusters=2, init='k-means++',
max_iter=100, n_init=1000, random_state=0)
kmeans.fit(df_sub_scaled)
```

```
#U datasetu gde se balaze skalirani podaci dodajemo novu
kolonu sa oznakama koja instanca pripada kom klasteru
df_sub_scaled["cluster"] = kmeans.labels_
df_sub_scaled
```

```
# Iscrtamo i uporedne boxplot-ove za ova dva klastera da bi
ih uporedili u smislu sadržaja (ovo je za 2 klastera)
fig, ax = plt.subplots(1,2) ## 1 se odnosi na br figura
koje cemo iscrtati a broj 2 je broj grafikona koji zelimo
da prikazemo u okviru slike (znc prikazuje jednu sliku sa
dva grafikona)
```

```
# U novijoj verziji matplotlib-a se umesto labels koristi
tick_labels
ax[0].boxplot(df_sub_scaled[df_sub_scaled["cluster"]==0].dr
op(columns = "cluster"))#, label=("sc", "bpm", "mod",
"enr", "liv")
ax[0].set_xlabel("Klaster 1")
```

```
# U novijoj verziji matplotlib-a se umesto labels koristi
tick_labels
ax[1].boxplot(df_sub_scaled[df_sub_scaled["cluster"]==1].dr
op(columns = "cluster"))
ax[1].set_xlabel("Klaster 2")
```

```
plt.show()
```

```
print("Klaster 1, N = ",
df_sub_scaled.loc[df_sub_scaled["cluster"]==0,
"cluster"].count())
print("Klaster 2, N = ",
df_sub_scaled.loc[df_sub_scaled["cluster"]==1,
"cluster"].count())
```

```
# Tumačenje
```

```
# Veličina klastera
```

```
# 1. Prvi klaster ima 402 pesme a drugi 550 pesama, oba su
slične veličine.
```

```
# 2. Centri klastera (gledamo gde je centar tj medijana)
```

*# U klasteru 1 se isključivo nalaze pesme u Minor modu (1)
a u klasteru 2 pesme u Major modu (0). Druga, doduše
minimalna razlika je to da je energy nivo malo niži za
pesme iz drugog klastera. Pitanje je da li je ovakva
klasterizacija naročito smisljena jer su po svim ostalim
karakteristikama pesme skoro iste u oba klastera.*

*# 3. Disperzija od centara #velicina ove kao tube
Za oba klastera možemo da kažemo da su skoro iste
homogenosti, jedina razlika je u kolonama energy i liveness
gde je klaster 1 malo homogeniji (manja disperzija).
Međutim, disperzija je inače jako velika u oba
klastera, što opet znači da bi možda trebalo probati sa
klasterizacijom sa četiri klastera.*

*# Posto cemo ponovo raditi klasterizaciju nad istim
podacima tj. istim DataFrame-om, moramo da izbacimo kolonu
cluster da ne utice na klasterizaciju*
`df_sub_scaled.drop(columns = ["cluster"], inplace=True)`

*# Radimo KMeans klasterizaciju sa četiri klastera i kmeans+
+ metodom za inicijalizaciju težišta (za 4)*
`kmeans = cluster.KMeans(n_clusters=4, init='k-means++',
max_iter=100, n_init=1000, random_state=0)
kmeans.fit(df_sub_scaled)`

*# U dataset u kojem se nalaze skalirani podaci
dodajemo novu kolonu sa oznakama koja instanca
pripada kojem klasteru*
`df_sub_scaled["cluster"] = kmeans.labels_`

*# Iscrtamo i uporedne boxplot-ove za ova četiri klastera da
bi ih uporedili u smislu sadržaja*
`fig, ax = plt.subplots(2,2)`

*# U novijoj verziji matplotlib-a se umesto labels koristi
tick_labels*
`ax[0,0].boxplot(df_sub_scaled[df_sub_scaled["cluster"]==0].
drop(columns = ['cluster']))
ax[0,0].set_xlabel("Klaster 1")`

```

ax[0,1].boxplot(df_sub_scaled[df_sub_scaled["cluster"]==1].
drop(columns = ['cluster']))
ax[0,1].set_xlabel("Klaster 2")

ax[1,0].boxplot(df_sub_scaled[df_sub_scaled["cluster"]==2].
drop(columns = ['cluster']))
ax[1,0].set_xlabel("Klaster 3")

ax[1,1].boxplot(df_sub_scaled[df_sub_scaled["cluster"]==3].
drop(columns = ['cluster']))
ax[1,1].set_xlabel("Klaster 4")

plt.show()

print("Klaster 1, N = ",
df_sub_scaled.loc[df_sub_scaled["cluster"]==0,
"cluster"].count())
print("Klaster 2, N = ",
df_sub_scaled.loc[df_sub_scaled["cluster"]==1,
"cluster"].count())
print("Klaster 3, N = ",
df_sub_scaled.loc[df_sub_scaled["cluster"]==2,
"cluster"].count())
print("Klaster 4, N = ",
df_sub_scaled.loc[df_sub_scaled["cluster"]==3,
"cluster"].count())

```

Tumačenje

Veličina klastera

1. Prvi klaster ima 173 pesme, drugi 245, treći 229 i četvrti 305, svi su značajne veličine.

2. Centri klastera

Klaster 1 sadrži pesme u Minor modu koje su loše rangirane na spotify top listi, dok klaster 2 sadrži pesme u Major modu koje su takođe loše rangirane na spotify top listi.

Nasuprot tome, klasteri 3 i 4 sadrže pesme koje su dobro rangirane na spotify top listi u Minor modu (klaster 3)

i Major modu (klaster 4) respektivno.

Razlika između dva klastera sa "dobro rangiranim" pesmama i dva klastera sa "loše rangiranim" pesmama je samo u tome

što dobro rangirane pesme (klasteri 3 i 4) imaju malo viši nivo energije (energy).

3. Disperzija od centara

Za klaster 1 i 2 se može reći da su izuzetno homogeni u smislu disperzije od centra klastera po atributu

in_spotify_charts jer ih isključivo čine loše rangirane pesme, dok su u klasterima 3 i 4 sve ostale pesme sa te top liste, pa je homogenost po tom atributu mnogo manja (odnosno std veća).

Za klaster 3 važi da je homogeniji od ostalih po atributima energy i liveness.

Ne postoje bitne razlike u homogenosti između klastera po ostalim atributima.

#%%

Hijerarhijska klasterizacija

```
#Neophodni importi
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
import sklearn.cluster as cluster

#Ucitavanje podataka
df = pd.read_csv('./podaci/spotify2.csv')
df.info()
display(df.head())

df.isna().sum()
#imaju in spotify charts i apple charts na vr
df_sub = df[df['speechiness']<=60].copy()
display(df_sub)
display(df_sub['speechiness'].describe())

#Biramo attribute koji ce biti ukljuceni u model
klasterizacije
df_sub.info()
#izbacujemo track_name jer je nerelevantna za rezultate
df_sub.drop(columns='track_name', inplace=True)
#spotify charts je str a treba int, mode je str
df_sub.describe()
# na osnovu toga vidimo da ce instrumentalness u delu od
25-75kv biti 0 pa je izbacujemo
df_sub.drop(columns='instrumentalness', inplace=True)
#mode ima dve vr major i minor tkd cemo je transf u
numericu

#transformacija kategorijskih u numericke
df_sub['in_spotify_charts'] =
pd.to_numeric(df_sub['in_spotify_charts'], errors='coerce')
df_sub.info()

#menjamo i kolonu mode ali njoj dajemo fixne vr 1 i 0
```



```
df_sub['mode'] = df_sub['mode'].map({'Major': 0, 'Minor': 1})
```

```
df_sub.info() #vidimo da smo promenili uspesno
```

```
df_sub.isna().sum()
```

```
#pos ukupno imamo samo 4 na vr onda cemo ih menjati pomocu medijane
```

```
df_sub.loc[df_sub['in_spotify_charts'].isna(),  
'in_spotify_charts'] = df_sub['in_spotify_charts'].median()
```

```
df_sub.loc[df_sub['in_apple_charts'].isna(),  
'in_apple_charts'] = df_sub['in_apple_charts'].median()
```

```
df_sub.isna().sum() #vidimo da smo uspesno promenili da nema na vr
```

```
#sad postoji problem a to je kodiranje podataka kojib ce uticati na klasterizaciju a to se ovde odnosi na prethodne dve kolone jer nmz d abude na nultom mestu
```

```
#onda se menja na mesto max
```

```
df_sub.loc[df_sub['in_spotify_charts'] == 0,  
'in_spotify_charts'] = df_sub['in_spotify_charts'].max()
```

```
df_sub.loc[df_sub['in_apple_charts'] == 0,  
'in_apple_charts'] = df_sub['in_apple_charts'].max()
```

```
df_sub.describe() #vidimo da se iz nule promenilo u 1
```

```
#matrica korelacije
```

```
corr_matrix = df_sub.corr(numeric_only=True).round(2)
```

```
plt.figure(figsize=(10, 8))
```

```
sb.heatmap(df_sub.corr(numeric_only=True), annot=True,  
           cmap='coolwarm', vmin=-1, vmax=1)
```

```
plt.title("Korelaciona matrica")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# u korelaciji su in_spotify_playlists, streams i in_apple_playlists pa ce to mng utiacti na reziltate klasterizacije stoga moramo izbaciti dva po zelji
```

```
df_sub.drop(columns=['in_apple_playlists', 'streams'],  
            inplace=True)
```

```
df_sub # vidimo da ih vise nema
```

```

#proveravamo broj outliera
# proveravamo outliere po kolonama jer oni uticu na
klasterizaciju, ako je vece od 10% onda se kolona izbacuje
a ako ih ima ali manje od 10% onda menjamo sa winsorize

#pravimo pomocnu funkciju draw_boxplot koja crta i pokazuje
boxplot
def draw_boxplot(podaci, naslov):
    plt.boxplot(podaci)
    plt.xlabel(naslov)
    plt.show()

from scipy.stats.mstats import winsorize

#trazimo outliere za jednu po jednu kolonu, izbacujemo,
menjamo
draw_boxplot(df_sub['released_year'], 'released year')
#ima outliere na donjoj granici pqa ce biti 0.10 do 0.0
winsorized_data = winsorize(df_sub['released_year'],
limits=[0.10, 0.0])
draw_boxplot(winsorized_data, 'Winsorized released year')
plt.show()
#ima vise od 10%
df_sub.drop(columns='released_year', inplace=True)

#skaliranje - normalizacija
from sklearn.preprocessing import StandardScaler,
MinMaxScaler

#promenljive su u veoma razl rasponima, potrebma je
normalizacija na skalu 0-1
#koristimo minmaxscaler koji vraca numpy array koji
pretvraamo nazad u dataframe

temp_data = MinMaxScaler().fit_transform(df_sub)
df_sub_scaled = pd.DataFrame(temp_data,
columns=df_sub.columns)
#provera kel sve skalirano kkao treba
df_sub_scaled.describe()

```

*#primena dendrograma utvrditi najbolju vrednost za broj klastera k i linkage metodu (udaljenost izmedju grupa)
#single – minimalno rastojanje najblizi par tacaka,
complete – max rastojanje, average – prosek svih parova,
ward – minimizuje porast ukupne avrijanse unutar klastera*

```
from scipy.cluster.hierarchy import linkage, dendrogram
```

```
#prikaz dendrograma za razl linkage metode  
dendrogram(linkage(df_sub_scaled, method='ward',  
metric='euclidean'))  
plt.title('Ward linkage')  
plt.show()
```

```
dendrogram(linkage(df_sub_scaled, method='single',  
metric='euclidean'))  
plt.title('Single linkage')  
plt.show()
```

```
dendrogram(linkage(df_sub_scaled, method='complete',  
metric='euclidean'))  
plt.title('Complete linkage')  
plt.show()
```

```
dendrogram(linkage(df_sub_scaled, method='average',  
metric='euclidean'))  
plt.title('Average linkage')  
plt.show()
```

*# Ako se izaberu complete ili average linkage metode,
dobija se
po 4 klastera koji su dosta "bliski" jedan drugom (visine
na
dendrogramu nisu prevelike). Sa single i Ward linkage
metodama,
dobijaju se po dva jasno odvojena klastera, sa tim da
Ward linkage
dozvoljava mogućnost podele na 4 klastera (razlika u
visini između 2 i 4
klastera nije velika).
Zbog svega navedenog, biramo Ward linkage metodu, a
probaćemo obe
varijante – podela na dva klastera i na četiri klastera.*

```

# izvršiti hijerarhijsku klasterizaciju za izabranu
vrednost k i linkage metodu
#radimo klasterizaciju za dva klastera i ward linkage
metodom, euklidska udaljenost
h_cluster = cluster.AgglomerativeClustering(n_clusters=2,
linkage='ward', metric='euclidean').fit(df_sub_scaled)

#u dataset u kojem se nalaze skalarni dodajemo novu kolonu
sa oznakama koja instanca pripada kojem klasteru

df_sub_scaled['cluster'] = h_cluster.labels_
display(df_sub_scaled)
df_sub_scaled.info()

#interpretirati dobijene klastere na osnovu broja pesama,
centara klastera, disperzije od centra

#iscrtamo i uporedne boxplotove za ova dva klastera da bi
ih uporedili u smislu sadržaja
fig, ax = plt.subplots(1,2)

ax[0].boxplot(df_sub_scaled[df_sub_scaled['cluster']==0].dr
op(columns='cluster'))
ax[0].set_xlabel("Klaster 1")

ax[1].boxplot(df_sub_scaled[df_sub_scaled['cluster']==1].dr
op(columns='cluster'))
ax[1].set_xlabel("Klaster 2")
plt.show()

print("Klaster 1, N = ",
df_sub_scaled.loc[df_sub_scaled['cluster']==0,
"cluster"].count())
print("Klaster 2, N = ",
df_sub_scaled.loc[df_sub_scaled['cluster']==1,
'cluster'].count())

#velicina klastera
#prvi klaster ima 550 pesama a drugi 402, oba slične
velicine
#centri klastera
# u klasteru 2 se isključivo nalaze pesme u Minor modu a u
klasteru 1 u Major. Druga minimalna razlika je da je energy
nivo u drugom klasteru malo viši nego u prvom. ne znamo da

```

li je klasterizacija nesto smislena jer su karakteristike skoro iste
#disperzija od centara
#za oba klastera kazemo da su skoro iste homogenosti,
jedina razlika je da je u kolonama energy za klaster 2 malo homogeniji, ali opet ne znamo nista

#klasterizacija sa 4 klastera

```
df_sub_scaled.drop(columns='cluster', inplace=True)
```

```
h_cluster = cluster.AgglomerativeClustering(n_clusters=4,  
linkage='ward', metric='euclidean').fit(df_sub_scaled)  
df_sub_scaled['cluster'] = h_cluster.labels_  
df_sub_scaled
```

```
fig, ax = plt.subplots(2,2)  
ax[0,0].boxplot(df_sub_scaled[df_sub_scaled['cluster']==0].  
drop(columns='cluster'))  
ax[0,0].set_xlabel('Klaster 1')
```

```
ax[0,1].boxplot(df_sub_scaled[df_sub_scaled['cluster']==1].  
drop(columns='cluster'))  
ax[0,1].set_xlabel('Klaster 2')
```

```
ax[1,0].boxplot(df_sub_scaled[df_sub_scaled['cluster']==2].  
drop(columns='cluster'))  
ax[1,0].set_xlabel('Klaster 3')
```

```
ax[1,1].boxplot(df_sub_scaled[df_sub_scaled['cluster']==3].  
drop(columns='cluster'))  
ax[1,1].set_xlabel('Klaster 4')
```

```
plt.show()
```

```
print("Klaster 1, N: ",  
df_sub_scaled.loc[df_sub_scaled['cluster']==0,  
'cluster'].count())  
print("Klaster 2, N: ",  
df_sub_scaled.loc[df_sub_scaled['cluster']==1,  
'cluster'].count())  
print("Klaster 3, N: ",  
df_sub_scaled.loc[df_sub_scaled['cluster']==2,  
'cluster'].count())
```

```
print("Klaster 4, N: ",  
df_sub_scaled.loc[df_sub_scaled['cluster']==3,  
'cluster'].count())
```

#velicina klastera

*# prvi klaster ima 304, drugi 231, treci 246, cetvrti 171.
svi su znacajne velicine*

#centri klastera

*#klasteri 1 i 3 sadrze pesme u major modu, a 2 i 4 u minor.
lose rangirane na spotify listi su 3 i 4 a dobro 1 i 2
Razlika između dva klastera sa "dobro rangiranim" pesmama
i dva klastera sa "loše rangiranim" pesmama je samo u tome
što dobro rangirane pesme (klasteri 1 i 2) imaju malo viši
nivo energije (energy).*

#disperzije od centara

*#za klastere 3 i 4 se moze reci da su vepma homogeni u
smislu disp od centara klastera po atributu
in_spotify_charts jer ih isključivo čine loše rangirane
pesme, dok su u klasterima 1 i 2 sve ostale pesme sa te top
liste, pa je homogenost po tom atributu mnogo manja
(odnosno std veća).
Za klaster 2 važi da je homogeniji od ostalih po
atributima energy i liveness.
Ne postoje bitne razlike u homogenosti između klastera po
ostalim atributima.*